VÕ TIẾN
Thảo luận kiến thức CNTT trường BK về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

Cấu Trúc Dữ Liệu và Giải Thuật (DSA)

DSA - HK243

Cuối Kì

Thảo luận kiến thức CNTT trường BK
về KHMT(CScience), KTMT(CEngineering)
https://www.facebook.com/groups/khmt.ktmt.cse.bku

TP. HỒ CHÍ MINH, THÁNG 7/2025

Dùng cho 10 câu sau

```
class Rope
{
public:
    static const int CHUNK_SIZE = 8;

private:
    class Node
    {
    public:
        enum BalanceFactor
        {
            LH = 1,
            EH = 0,
            RH = -1
        };
        friend class Rope;

    private:
        Node *left;
        Node *right;
        string data;
        int weight;
        int height;
        BalanceFactor balance;

        Node();
        explicit Node(const string &s);
        bool isLeaf() const;
        friend class TestHelper;
    };

    Node *root;

    int height(Node *node) const;
    int getTotalLength(Node *node) const;
    void update(Node *node);
    Node *rotateLeft(Node *x);
    Node *rotateRight(Node *y);
    Node *rebalance(Node *node);
    void split(Node *node, int index, Node *&outLeft, Node *&outRight);
    Node *concatNodes(Node *left, Node *right);
    char charAt(Node *node, int index) const;
    string toString(Node *node) const;
    void destroy(Node *&node);

public:
    Rope();
    ~Rope();

    int length() const;
    bool empty() const;
    char charAt(int index) const;
    string substring(int start, int length) const;
    void insert(int index, const string &s);
    void deleteRange(int start, int length);
    string toString() const;
};
```

1. Điền code vào đoạn còn trống **CODE 1 và CODE 2**

```
Node *rotateLeft(Node *x)
{
    Node *y = x->right;
    /* CODE 1 */
    y->left = x;
    /* CODE 2 */
    // TODO
    return y;
}
```

   a) `x->right = y->left; updateHeight(x);`
   b) `x->right = y; updateHeight(y);`
   c) `x->left = y->right; updateHeight(x);`
   d) `y->left = x->right; updateHeight(y);`

   Đáp án: A

2. Điền code vào đoạn còn trống **CODE 1 và CODE 2** trong hàm `rebalance`:

```
Node *rebalance(Node *node)
{
    if (node->balance == Node::LH)
    {
        if (// TODO)
            /* CODE 2 */
        if (// TODO)
            return rotateRight(node);
    }
    else if (node->balance == Node::RH)
    {
        if (// TODO)
            node->right = rotateRight(node->right);
        if (/* CODE 1*/)
            // TODO
    }
    return node;
}
```

   a) `node->left = node->left->balance == Node::RH;`
      `node->right = rotateRight(node->right);`
   b) `node->left = node->left->balance == Node::LH;`
      `node->left = rotateRight(node->left);`
   c) `node->right = node->right->balance == Node::LH;`
      `node->right = rotateLeft(node->right);`
   d) `node->right = node->right->balance == Node::RH;`
      `node->left = rotateLeft(node->left);`

   Đáp án đúng: D

3. Điền code vào đoạn còn trống trong hàm `charAt` của cấu trúc Rope:

```
char charAt(Node *node, int index) const
{
    if (node->isLeaf())
    {
        /* CODE 1 */
    }
    if (/* CODE 2 */)
```

```
        // TODO
    else
        return charAt(node->right, /* CODE 3 */);
}
```

a) `return node->data[index]; index < node->weight; index - node->weight`
b) `return node->data[index]; index > node->weight; index + node->weight`
c) `return node->data[index]; index <= node->weight; index - node->weight`
d) `return node->data[index]; index >= node->weight; index + node->weight`

Đáp án đúng: A

4. Xét đoạn code sau dùng vòng `for` để tìm ký tự đầu tiên xuất hiện trong Rope có dạng:

```
char findFirstChar(const Rope &rope, char target) {
    int n = rope.length();
    for (int i = 0; i < n; ++i) {
        if (rope.charAt(i) == target) {
            return rope.charAt(i);
        }
    }
    return '\0'; // Không tìm thấy
}
```

Độ phức tạp thời gian của hàm `findFirstChar` là bao nhiêu?

a) $\mathcal{O}(n)$ b) $\mathcal{O}(n \log n)$ c) $\mathcal{O}(\log n)$ d) $\mathcal{O}(1)$

Đáp án: B

5. Điền đoạn code còn thiếu **CODE 1** trong hàm `substring` của cấu trúc Rope.

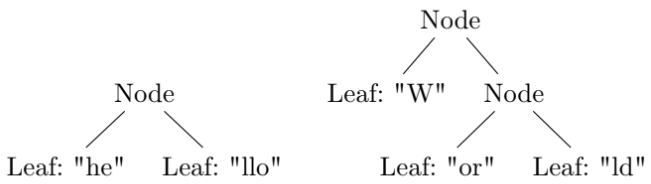
```
string substringHelper(Node *node, int start, int end) const
{
    // TODO
    if (end < leftWeight)
    {
        // TODO
    }
    else if (start >= leftWeight)
    {
        // TODO
    }
    else
    {
        return /* TODO */ + /* CODE 1 */;
    }
}

string Rope::substring(int start, int len) const
{
    return substringHelper(root, start, start + len - 1);
}
```

a) `substringHelper(node->right, 0, end - leftWeight);`
b) `substringHelper(node->right, 0, end - leftWeight - 1);`
c) `substringHelper(node->left, start, leftWeight - 1`
d) `substringHelper(node->left, start, leftWeight`

Đáp án đúng: A

6. Cho hai cây Rope biểu diễn chuỗi như sau:



Sau khi gọi hàm `concatNodes(rootA, rootB)` tạo node gốc mới, giá trị `weight` và `height` của node gốc là bao nhiêu?

a) weight = 5, height = 4
b) weight = 10, height = 5
c) weight = 5, height = 3
d) weight = 10, height = 2

Đáp án đúng: A

Hỏi 3 câu sau

```
void Rope::split(Node *node, int index, Node *&outLeft, Node *&outRight)
{
    // TODO

    if (node->isLeaf())
    {
        if (index <= 0) // TODO
        else if (index >= (int)node->data.size())
        {
            outLeft = node->data;
            outRight = nullptr;
        }
        else
        {
            outLeft = // TODO;
            outRight = /* CODE 1*/;
        }
        return;
    }
    if (index < node->weight)
    {
        // TODO
    }
    else if (index > node->weight)
    {
        Node *rightL = nullptr, *rightR = nullptr;
        /* CODE 2*/
        /* CODE 3*/
        outRight = rightR;
    }
    else
    {
        outLeft = node->left;
        outRight = node->right;
    }
    delete node;
}
```

7. Điền code vào **CODE 1** trong hàm `split` của Rope:

a) `node->data = node->data.substr(index); node->weight = node->data.size()`
b) `new Node(node->data.substr(0, index))`

c) `new Node(node->data)`
d) `new Node(node->data.substr(index + 1, node->data.size() - index))`

Đáp án: A

8. Điền code vào **CODE 2** trong hàm `split` của Rope:

a) `split(node->right, index - node->weight, rightL, rightR);`
b) `split(node->right, index, rightL, rightR);`
c) `split(node->left, index, rightL, rightR);`
d) `split(node->left, index - node->weight, rightL, rightR);`

Đáp án: A

9. Điền code vào **CODE 3** trong hàm `split` của Rope:

a) `outLeft = concatNodes(node->left, rightL);`
b) `outLeft = concatNodes(rightL, node->left);`
c) `outLeft = concatNodes(node->right, rightL);`
d) `outLeft = concatNodes(rightR, node->right);`

Đáp án: A

10. Biểu diễn cây khi khi thực hiện bước sau

```
Rope rope;
rope.insert("Hello_World");
rope.deleteRange(5, 5);
rope.insert("_DataStructure");
```

a) `17 (11 (3 ("Hel") ("_DataStr")) ("cture")) (2 "lo") ("d"))`
b) `17 (11 (3 ("Hel") ("_DataStr")) ("cture")) ("lod"))`
c) `17 (11 ("Hel_DataStr") ("cture")) (2 "lo") ("d"))`
d) `20 (11 (3 ("Hel") ("_DataStr")) ("cture")) (2 "lo") ("d"))`

Đáp án: A